

Локальные и облачные AI-инструменты разработчика

Что считать у себя на ноутбуке, а что доверить чужому дата-центру



Макеев Владимир
Старший разработчик информационных систем
tg: @vv_makeev



AI уже не игрушка, а часть профессии

84%

разработчиков уже используют
AI-инструменты или
планируют начать

51%

профессиональных
разработчиков обращаются к
AI ежедневно

ВНИМАНИЕ

29%

из них доверяют точности того,
что выдает AI

Источник: Stack Overflow Developer Survey

Инструментом пользуются почти все, а доверяют ему меньше трети. Выбор инструмента — это управление риском, а не вопрос удобства.

План на 45 минут — четыре блока

01

Как это устроено

Архитектура, локально / self-hosted / облако, жанры инструментов

02

Облако

Copilot, агентские системы, Claude Code, чат-модели, API, MCP

03

Локально

Модели, Ollama, железо

04

Как выбирать

Матрица решений, гибрид, тренды 2026





01

Как это устроено

- Архитектура и веса модели
- Локально, self-hosted, облако
- Жанры AI-инструментов

Один принцип, два способа его запустить

- 1 Токенизация.** текст режется на токены — слово, часть слова или символ
- 2 Предсказание токена.** модель считает вероятности: какой токен дальше
- 3 Контекстное окно.** модель помнит только то, что влезло в окно — весь код, который вы ей показали
- 4 Повтор.** цикл повторяется, пока не соберется полный ответ

Что такое веса модели

Числа, которые нейросеть выучила при обучении. Архитектура — это устройство мозга, веса — сила конкретных связей. Файл с весами весит десятки гигабайт.

Трансформер трансформеру рознь

Copilot и Qwen оба используют архитектуру трансформер, но внутри она разная: dense-модель считает всеми параметрами сразу, MoE-модель — только частью. Подробнее — в блоке про локальные модели.

Три места, где может жить модель



У вас на диске

Веса скачаны на ваш GPU. Считает ваша видеокарта или Apple Silicon.

Владелец весов: вы

Считает: ваше железо

Интернет: не нужен



Self-hosted в компании

Открытая модель на серверах компании: vLLM или SGLang раздают ее всей команде разработчиков.

Владелец весов: компания

Считает: сервер компании

Интернет: только внутри сети



В чужом дата-центре

Запрос уходит провайдеру, там его считает GPU-кластер, ответ приходит обратно.

Владелец весов: провайдер

Считает: дата-центр провайдера

Интернет: обязателен

Жанр 1. Автодополнение



```
hello.go > main
1 package main
2
3 import "fmt"
4
5 func main() {
6     fmt.Println("Hello, World!")
7 }
8
9
```

Autocomplete dropdown for line 6:

- `fmt` (selected)
- `fmt.Append`
- `fmt.Appendf`
- `fmt.Appendln`
- `fmain` (main function)

Подсказывает следующую строку или блок прямо во время набора текста.

Что вы делаете

Нажимаете Tab, чтобы принять подсказку, или продолжаете печатать сами.

Жанр 2. Чат-ассистент в IDE

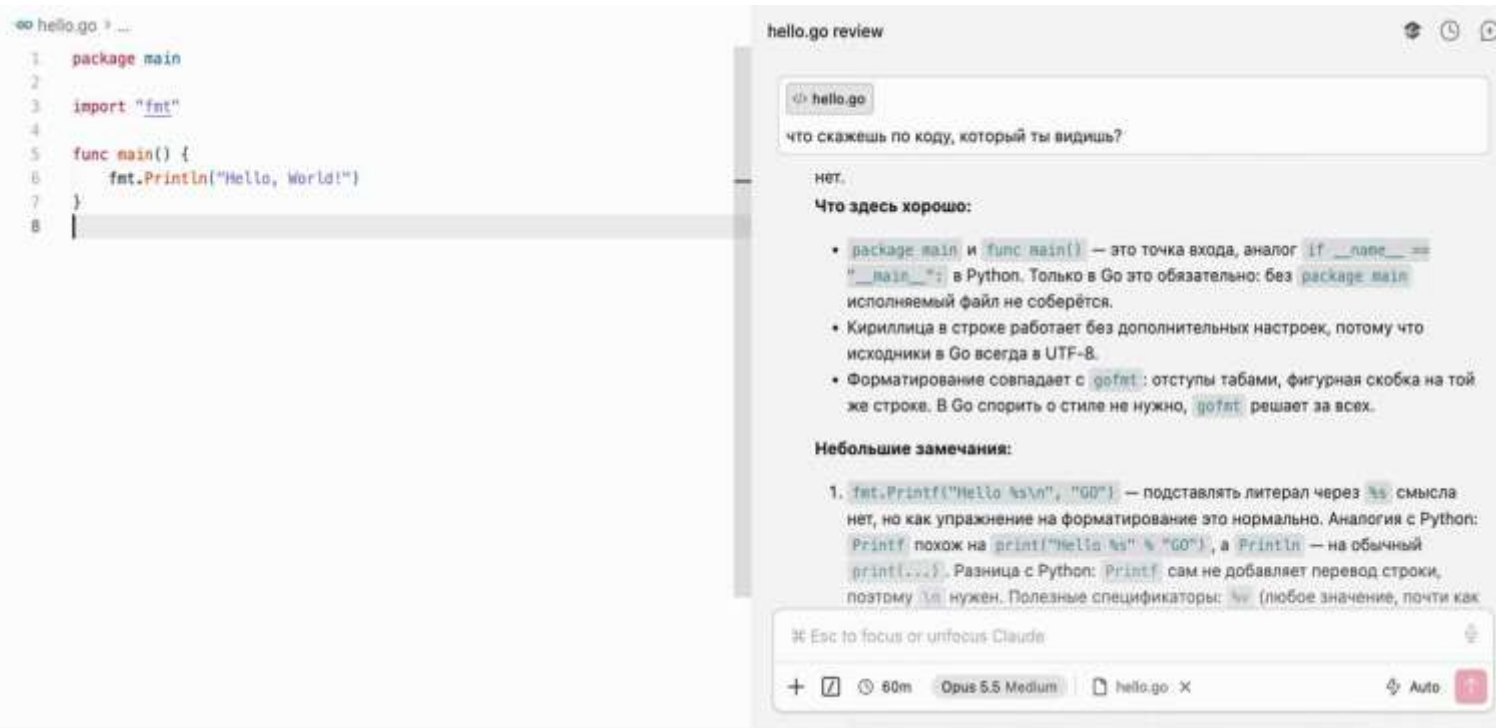


Боковая панель, где можно спросить, объяснить код или попросить отрефакторить кусок.

Что вы делаете

Пишете вопрос или задачу текстом, читаете ответ, сами решаете, вставлять код или нет.

Отличие от жанра 1: инициатива у вас, модель не действует сама.



Жанр 3. Агент: LLM

Агент работает по циклу



Думает
planning



Действует
tool call



Наблюдает
результат

Шаг повторяется,
пока задача не
будет решена
целиком.



Сам читает файлы, правит несколько сразу, запускает команды и итерирует.

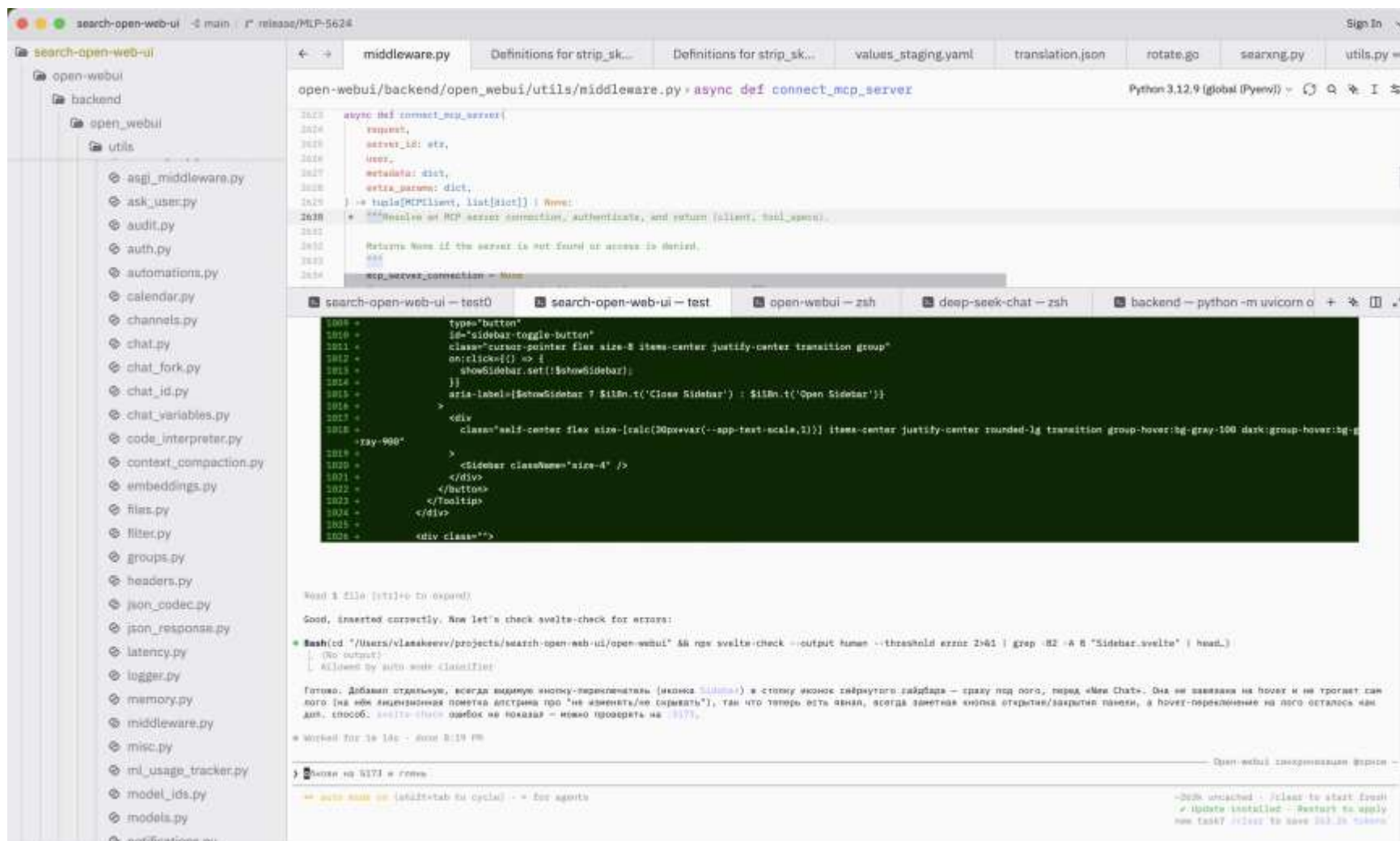
Из чего состоит агент

LLM-ядро думает, решает, что делать дальше

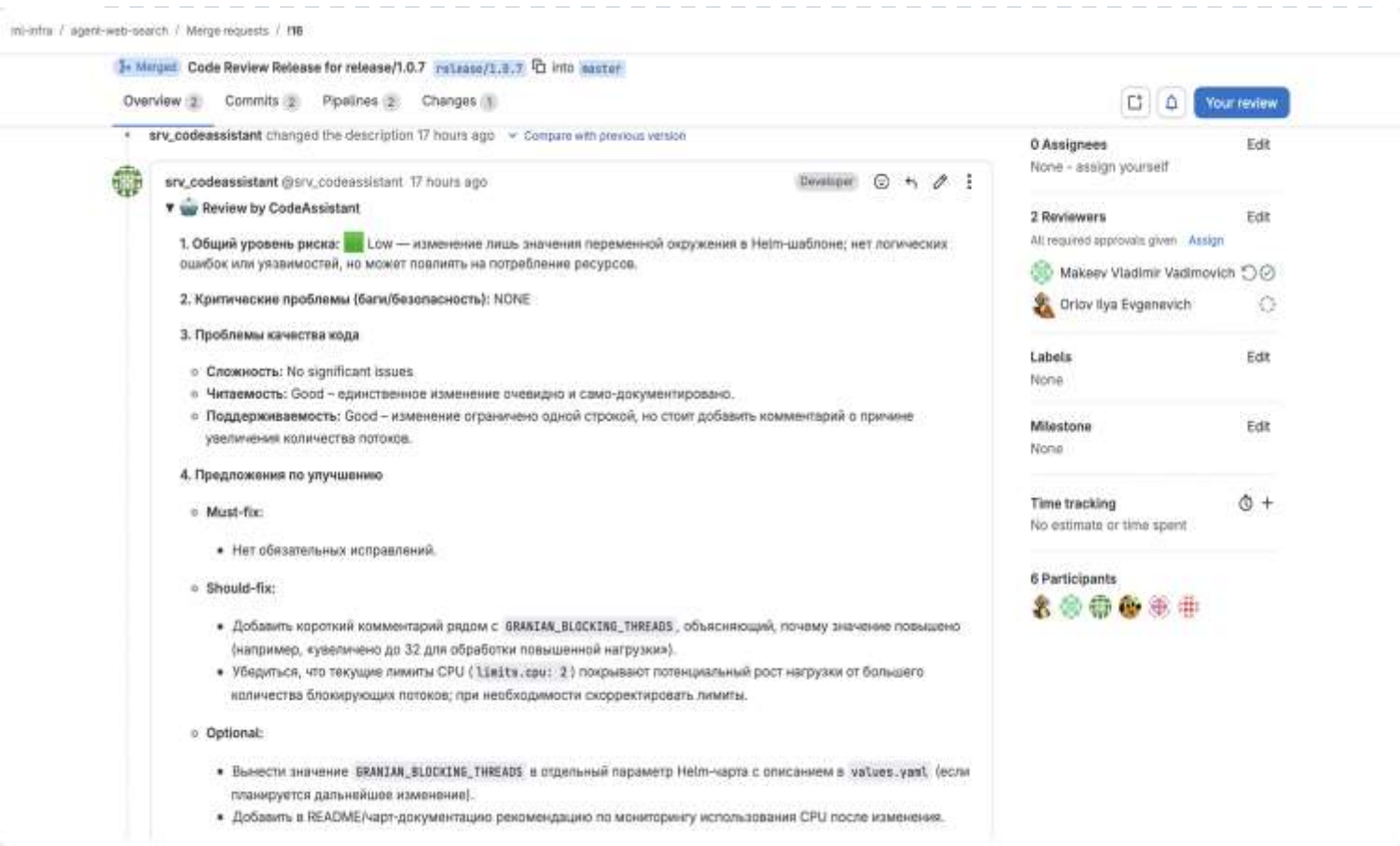
Инструменты файлы, shell, поиск, git

Память история действий в контексте

Жанр 3. Агент: LLM



Жанр 4. Агент на Git-хостинге



Тот же агент, но живет не у вас в терминале, а на сервере GitHub или GitLab.

Что делает

Сам приходит в Pull Request: ищет баги, проверяет стиль, объясняет риски изменений ревьюеру.

В чем разница

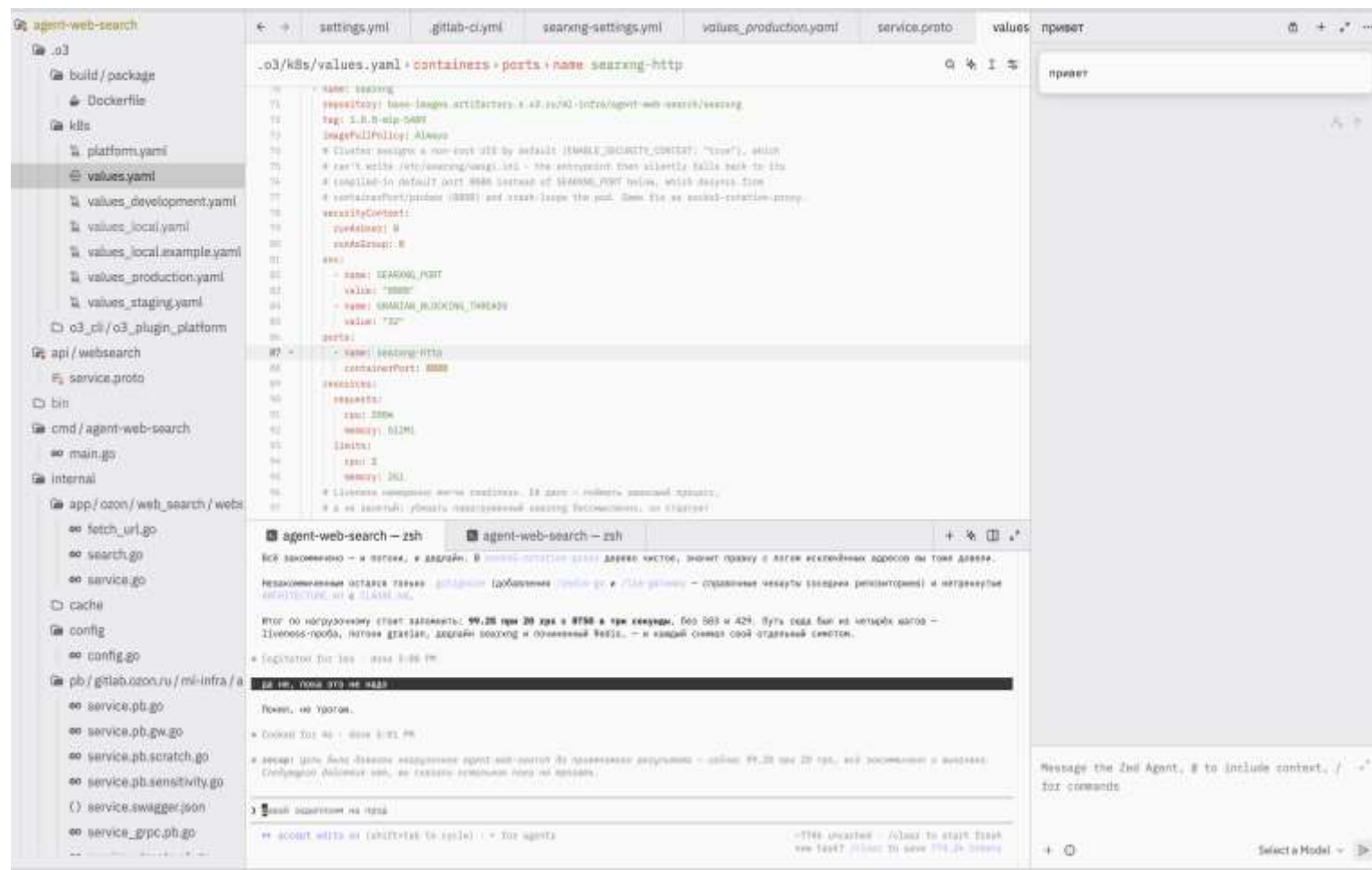
Жанр	Кто инициатор действия	Что вы получаете	Автономность
Автодополнение	Вы	Строка или блок кода	Только смотрит на текущий файл
Чат-ассистент	Вы	Ответ на вопрос в чате	Отвечает, но сам не действует
Агент	Модель	Измененные файлы, команды, коммит	Читает, правит, запускает, проверяет

02

Облако

- GitHub Copilot
- Claude Code и агентские системы
- Чат-модели, API, MCP

Zed: редактор нового поколения



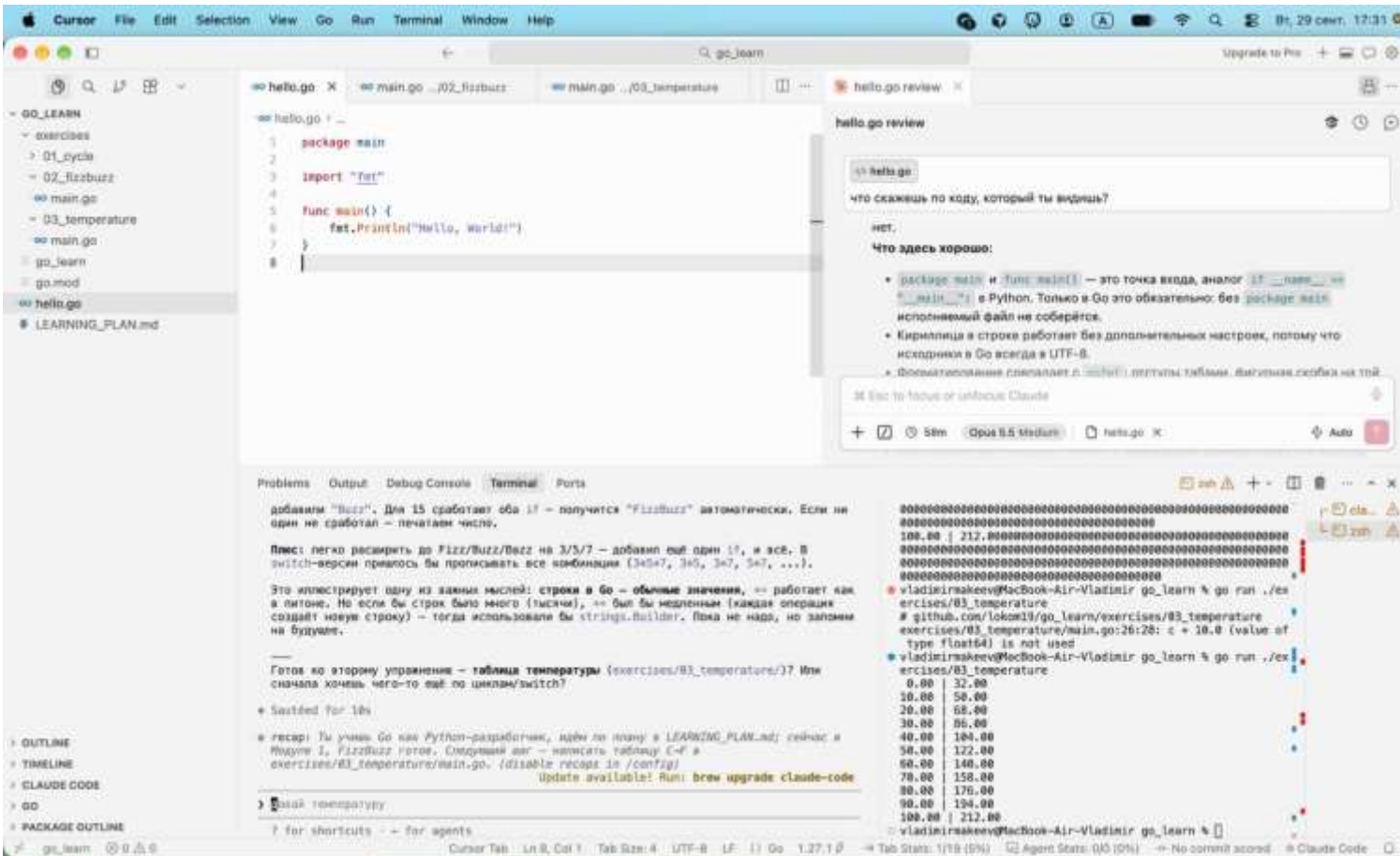
Не плагин к существующей IDE, а отдельный редактор, написанный с нуля ради скорости.

Что внутри

AI-функции, автодополнение и чат, встроены нативно, а не поверх старого редактора.

Плюс: очень быстрый отклик интерфейса и совместное редактирование в реальном времени.

Cursor: автодополнение и чат в одном окне



Форк VS Code: интерфейс и расширения знакомые, но AI встроен глубже, чем в виде плагина.

Два режима сразу

Автодополнение по Tab, как у Copilot, и отдельная чат-панель сбоку, которая видит весь проект и может менять сразу несколько файлов.

Claude Code: терминальный агент

```
$ claude
```

```
> читаю структуру проекта...
```

```
> нашел баг в auth/session.ts
```

```
> правлю 3 файла, запускаю тесты...
```

```
> npm test — 42 passed
```

```
> готово. Commit создан.
```

Никакого графического интерфейса. Вы даете задачу, агент сам читает, правит и проверяет, пока она не решена.

Claude Code (Anthropic)

Одно из самых больших контекстных окон в классе и один из лучших результатов на бенчмарке SWE-bench. Берут для по-настоящему сложных задач.

МСП из коробки

Родная интеграция с внешними системами через открытый протокол — подробнее через два слайда.

Агенту дан доступ к терминалу — ограничивайте права и ставьте ревью перед merge.

Тот же класс инструментов: **Cursor** и **Windsurf** встраивают агента прямо в редактор, **OpenCode** — открытый терминальный агент, который можно подключить и к облаку, и к локальной модели.

Чат-модели и API

Тот же API можно вызвать из своего скрипта, а не только открыть чат в браузере.

Чат в браузере

быстро спросить, объяснить ошибку, накидать план

API + свой код

вызываете модель из своего скрипта или сервиса

RAG-паттерн

модель ищет ответ по вашей базе знаний, а не только по памяти

Пример: «API + свой код»

Пишете скрипт на Python или Node, который отправляет запрос к модели через ее API: текст промпта плюс ключ доступа. Модель возвращает ответ в виде текста, скрипт обрабатывает его дальше — сохраняет в файл, вставляет в другую программу, отправляет в Telegram-бота.

RAG

Retrieval augmented generation: перед ответом модель сначала находит нужные фрагменты в вашей базе знаний, а не отвечает только по памяти.



Главное отличие от обычного чата: ответ модели опирается на конкретные документы, а не только на то, что она запомнила при обучении, поэтому данные можно обновлять без переобучения модели.

MCP



Открытый протокол, который подключает модель к сервисам и данным: файлам, базам, Confluence, GitHub.

Tool

вызвать функцию по имени

`создать_страницу()`

Resource

прочитать источник данных

`confluence://страница`

Skill

выполнить готовый сценарий

`сценарий ревью кода`

Без MCP

Модель отвечает из головы. Данные устаревшие. Только текст, без действий.

С MCP

Модель подтягивает живые данные и может изменять ресурсы. Текст плюс действия.

MCP на примере: поиск страницы в Confluence

Допустим, ваш MCP-сервер дает модели инструмент `search_confluence_page(query)`. Разберем, что происходит от вопроса до ответа.

- 1 Вопрос.** сотрудник спрашивает у агента, где посмотреть политику отпусков
- 2 Модель видит инструмент.** среди доступных MCP-инструментов есть `search_confluence_page` с описанием, что он делает
- 3 Модель решает его вызвать.** формирует вызов с параметром `query`, текст запроса модель придумывает сама
- 4 MCP-сервер выполняет запрос.** обращается к реальному Confluence по API и возвращает найденный текст модели
- 5 Модель отвечает.** собирает ответ на основе полученного текста страницы и может приложить ссылку

Что уходит туда и обратно

```
→ tool_call:  
  search_confluence_page(  
    query: "политика отпусков"  
  )  
  
← result:  
  title: "Отпуска и больничные"  
  url: "wiki/HR/leave-policy"  
  text: "Отпуск оформляется..."
```

Модель сама не умеет ходить в Confluence: это делает MCP-сервер, у которого есть реальный доступ и права. Модель только просит и получает готовый результат.

Плюсы облачного подхода



Всегда самая новая модель — обновляется без вашего участия



Ноль настройки: открыл и работает



Считает провайдер, вашей машине не нужны мощности



За инфраструктуру и безопасность отвечает не вы



Работает с любого устройства



Богатая экосистема интеграций через MCP

Минусы и риски



Apple запретила сотрудникам ChatGPT и Copilot

опасаясь утечки внутренних данных



Обучение на вашем коде по умолчанию

бесплатный тариф Copilot с апреля 2026 отправляет код на дообучение, если это не отключить



Оплата по факту использования

с июня 2026 Copilot перешел на usage-based биллинг



Инциденты безопасности случаются

баг в Copilot читал защищенные письма в обход DLP-политик



Нужна сеть, и вы зависите от провайдера

политика или цены могут измениться в любой момент

03

Локально

- Зачем считать у себя
- Какую модель ставить
- Ollama, llama.cpp, железо

Локально: зачем и когда

Данные не покидают контур

код, база, переписка — все остается на вашей машине или в вашей сети

Работает без интернета

в поезде, в самолете, в изолированном контуре без доступа к облаку

Платите один раз за железо

дальше нет счетчика токенов — можно гонять модель сколько угодно

Пример: во сколько раз квантование сжимает модель

FP16 — полная точность

70B параметров × 2 байта ≈ 140 ГБ

нужно 2+ видеокарты уровня A100

Q8 — 8-битное квантование

≈ 70 ГБ

влезает в одну мощную карту

Q4 — 4-битное квантование

≈ 35 ГБ

запускается на игровом ноутбуке с 32+ ГБ памяти

Меньше бит на параметр — меньше памяти и места на диске, но точность ответов немного падает.

Какую модель поставить

Открытые веса — это модели, чьи веса можно скачать и запускать у себя.

Llama (Meta)

самое широкое сообщество и больше всего готовых инструментов вокруг

Qwen (Alibaba)

сильные результаты в коде и математике, много размеров под разное железо

DeepSeek

MoE-архитектура — быстрая генерация при большом числе параметров

Mistral / Gemma

компактные модели, которые уверенно работают на одном GPU

Чем запускать



Ollama

одна команда — и модель скачана, запущена и доступна по локальному API

```
ollama run qwen2.5-coder
```



llama.cpp

движок, на котором Ollama работает изнутри; ниже уровнем, больше контроля над памятью и скоростью

```
./llama-server -m model.gguf
```

На практике запуск проще, чем кажется: не сложнее установки обычной программы. LM Studio и Jan решают ту же задачу через графический интерфейс, но за консолью проще понять, что происходит внутри.

Тот же плагин в IDE, просто другой адрес

Ollama поднимает локальный сервер, который отвечает так же, как облачный API. Многие плагины для IDE умеют переключаться на него одной настройкой.

```
# Настройка плагина в IDE
```

```
api_base: http://localhost:11434/v1    # было: api.openai.com
```

```
model: qwen2.5-coder:14b
```

```
api_key: не требуется
```

Код плагина не знает и не спрашивает, где физически стоит модель. Автодополнение и чат работают так же, как с облачным провайдером — просто запрос уходит на localhost, а не в интернет.

Железо: что реально нужно

VRAM / память	Пример железа	Модель	Что получаете
8 ГБ VRAM	RTX 3060/4060, база Mac 16 ГБ	7–9B, Q4	лёгкий чат, простые подсказки
12–16 ГБ VRAM	RTX 4070/4080	14B комфортно, 32B впритык	уверенный код-ассистент
24 ГБ VRAM	RTX 4090 / 3090	32B-coder, Q4, ~70 ток/с	замена Copilot по автодополнению
48 ГБ+ / Apple 64–128 ГБ	workstation, Mac Studio	70B+ или MoE 30B–A3B	серьёзная рабочая лошадка

Плюсы локального подхода



Полный контроль над данными

ничего не уходит за пределы вашей машины или сети



Нет счетчика токенов

заплатили за железо один раз — дальше считайте сколько угодно



Работает офлайн

поезд, самолет, изолированный контур без доступа в интернет



Можно дообучить под себя

подстроить модель под свой стиль кода и внутренние соглашения

Минусы и риски локального подхода



Нужно железо, и оно стоит денег

мощная видеокарта — от 150–200 тысяч рублей единовременно



Модели слабее топовых облачных

открытые веса догоняют, но на сложных задачах пока отстают от лучших закрытых моделей



Настройка и обновления — на вас

никто не подкрутит сервер за вас, вы сами следите за версиями



Меньше контекстное окно

локально сложно держать такой же большой контекст, как в топовых облачных моделях

04

Как выбирать

- Матрица решений
- Гибридный подход
- Тренды 2026

Матрица решений

Ситуация	Что выбрать	Почему
Прототип, эксперимент	Облако	нулевая настройка, всегда свежая модель
Код под NDA / гостайна	Локально	данные не должны покидать контур
Изолированная сеть без интернета	Локально	единственный вариант, который вообще работает
Команда, много параллельных задач	Облако	не упираетесь в железо одной машины
Массовая обработка кода компании	Self-hosted в своей VPC	баланс контроля и масштаба

Гибридный подход

В реальных командах локальное, облачное и self-hosted обычно сосуществуют — выбор делается под конкретную задачу, а не под всю компанию сразу.

Черновой код и автодополнение

облачный подход — скорость
важнее контроля

Работа с закрытым кодом

локальная модель на ноутбуке
разработчика/self-hosted

Массовое код-ревью в CI/CD

self-hosted vLLM в своей VPC —
масштаб и комплаенс

Тренды 2026: куда все движется



Открытые веса догоняют закрытые модели

Qwen и DeepSeek на многих бенчмарках почти на уровне топовых закрытых моделей



Агентные системы становятся стандартом

от автодополнения к моделям, которые сами планируют и выполняют многошаговые задачи



MCP закрепился как общий протокол

все больше сервисов подключаются к моделям по одному стандарту, а не по своим API



Self-hosted serving становится проще

vLLM и SGLang снижают порог входа для компаний, которым нужен контроль над данными

ИТОГИ

- ✓ Автодополнение, чат, агент и агент на git-хостинге — не конкуренты, а разные уровни автономности одного и того же инструмента
- ✓ Локально — контроль и офлайн-доступ; облако — мощность и удобство; self-hosted — попытка взять лучшее от обоих
- ✓ Выбор — это управление риском под конкретную задачу, а не разовое решение на всю карьеру

Вопросы?